Eduvos Assessment Coversheet

Academic Details

Campus:	**Eduvos Online**
Faculty:	**Information Technology**
Module Code:	**ITBDA4-14**
Lecturer's Name:	**Mr Gilbert Lekula (Ngobeni)**
Group (if applicable)	**N/A**

Assessment Details

Assessment Type:	**Project 2**
Assignment/Brief Number:	**2**
Due Date/ Submission Date:	**13 Nov 2026**

Student Details (Group, if applicable)

#	Full Name	Student Number	Contact Number	% Participation	Signature
1					
2					
3					
4					
5					
6					

Student Details (Individual, if applicable)

Student Full Name:	**Happy Mahlangu**
Student Number:	**EDUV13317839**

Academic Integrity

AI Disclosure Completed	**X** **Yes**	**No**

Declaration:
I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature	**H.Mahlangu**
Date	**13 Nov 2026**

Lecturer Use Only

Lecturer Comments:

Marks Awarded (%):	
Lecturer Signature:	
Date:	

Assessment

Table of Contents

Introduction **4**

Question 1: Hadoop MapReduce keyword search **5**

1.1 Logical layout of the keyword search 5

1.2 Mapper implementation 6

1.3 Reducer implementation 11

1.4 Hadoop MapReduce configuration 12

Question 2: Topic modelling with LDA **16**

2.1 Data frame and denoising 16

2.2 LDA topics and word distributions 17

2.3 Visualising the top words per topic 19

Question 3: WebText API proof of concept **24**

3.1 Extracting clean web text into an R data structure 24

3.2 Preprocessing 25

3.3 Text summarisation 26

Question 4: ETL with R and PostgreSQL **34**

4.1 Use case and scraping 34

4.2 Cleaning, denoising and imputation 36

4.3 Loading into PostgreSQL and analysis 37

4.4 GitHub repository 40

References **42**

Introduction

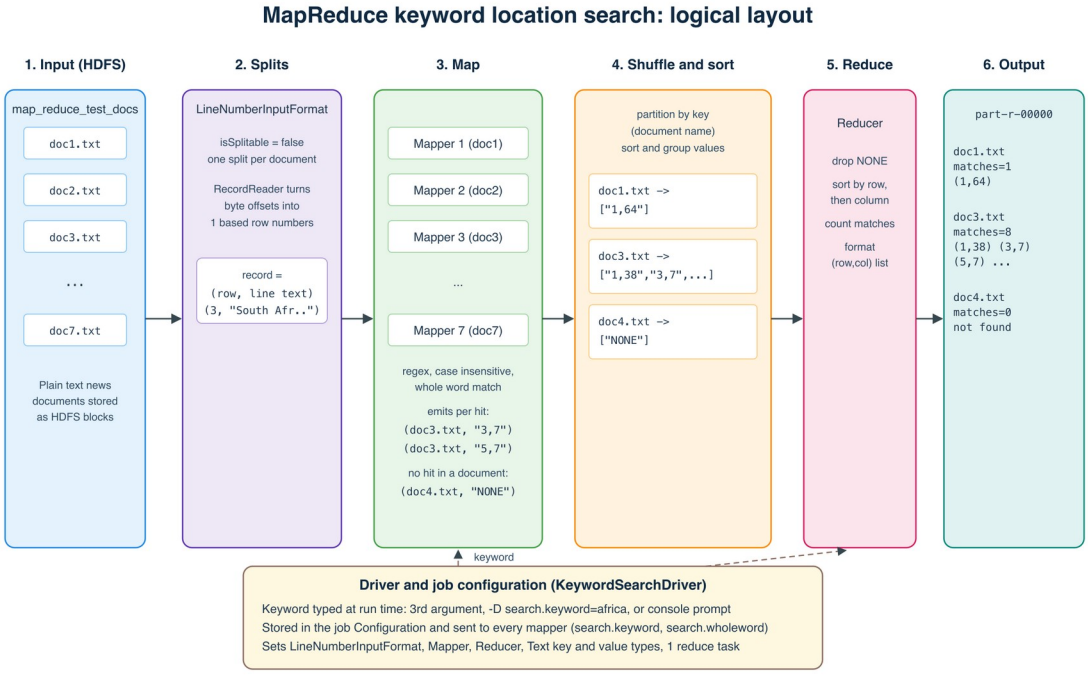
This report answers the four questions of Project 2. Every answer comes with working code that was run on the datasets provided or on data scraped for this project. The full project, including the datasets, the Java MapReduce project, the R Markdown notebooks and their rendered HTML output, is in the private GitHub repository **https://github.com/HappyDevs1/big-data-text-mining-etl** and in the zipped code submission.

Question	Folder	Main files
1. Hadoop MapReduce keyword search	`q1-mapreduce/`	Java source, Hadoop config, run scripts, `evidence/`
2. LDA topic modelling	`q2-lda/`	`q2_lda_topic_modelling.Rmd` and `.html`
3. WebText API proof of concept	`q3-webtext/`	`q3_webtext_mining.Rmd` and `.html`
4. ETL with R and PostgreSQL	`q4-etl/`	`q4_etl_books.Rmd` and `.html`, `sql/`
Datasets	`data/`	`raw/`, `extracted/`, `processed/`

Tools used: Java 17, Apache Hadoop 3.5.0 (HDFS and YARN on a single node), Maven, R 4.6 with tidytext, topicmodels, textrank, lexRankr, rvest and RPostgres, and PostgreSQL 17.

Question 1: Hadoop MapReduce keyword search

1.1 Logical layout of the keyword search



Logical layout of the MapReduce keyword location search

The goal is to return the row and column of every occurrence of a keyword in every document. The keyword is not fixed in the code, and matching ignores case. The job runs in six logical stages.

1. **Input.** The seven documents from `map_reduce_test_docs.zip` are copied into HDFS (`/user/<user>/keyword-search/input`). HDFS stores them as blocks spread across DataNodes.
2. **Input splits and record reading.** Hadoop's default `TextInputFormat` hands the mapper a byte offset, which is not a row number. Rows can only be counted correctly if one reader sees a document from its first line. A custom `LineNumberInputFormat` therefore marks files as not splittable, so each document becomes one split. Its record reader wraps Hadoop's `LineRecordReader` and replaces the byte offset with a running line counter. Every record becomes `(rowNumber, lineText)`. Lines are still streamed one at a time, so memory use stays flat for large documents.
3. **Map.** One mapper runs per document, in parallel across the cluster. In `setup()` it reads the keyword from the job configuration and compiles a case insensitive, whole word regular expression. For each line it finds every match and emits `(documentName, "row,column")`, where the column is the 1 based character position where the match starts. If a document has no match at all, the mapper emits `(documentName, "NONE")` in `cleanup()` so that the result also lists documents where the word was not found. Hadoop counters record lines scanned, matches and documents with a match.
4. **Shuffle and sort.** The framework partitions the map output by key (the document name), sends each key to a reducer, and groups all of a document's locations together.

5. **Reduce.** The reducer receives `(documentName, [locations])`, drops the `NONE` marker, sorts the locations by row and then by column, counts them and formats one output line per document.
6. **Output.** With one reduce task the result is a single file, `part-r-00000`, for example `doc3.txt keyword=africa  matches=8  locations=(1,38) (3,7) (5,7) ....`

The **driver** ties it together. It reads the keyword from the third command line argument, a `-D search.keyword=` option, or a console prompt, puts it in the job `Configuration` (which Hadoop ships to every task), and wires up the input format, mapper, reducer and key and value types.

1.2 Mapper implementation

The mapper relies on two helper classes: the input format that supplies row numbers, and a small matcher class that does the case insensitive search. Keeping the matching logic in its own class means it can be unit tested without Hadoop (`KeywordMatcherTest`, 4 tests passing).

LineNumberInputFormat.java gives the mapper `(row, line)` instead of `(byteOffset, line)`:

```
package za.itbda.keywordsearch;

import java.io.IOException;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.InputSplit;
import org.apache.hadoop.mapreduce.JobContext;
import org.apache.hadoop.mapreduce.RecordReader;
import org.apache.hadoop.mapreduce.TaskAttemptContext;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.LineRecordReader;

/**
 * Input format that hands the mapper (lineNumber, lineText) pairs.
 *
 * The default TextInputFormat gives a byte offset as the key, which is not a
 * row number. To know the real row of every line, each document must be read
 * from its first line by a single mapper, so files are marked as not splittable.
 * Lines are still streamed one at a time, so memory use stays flat even for big
 * documents.
 */
public class LineNumberInputFormat extends FileInputFormat<LongWritable, Text> {

    @Override
    protected boolean isSplitable(JobContext context, Path file) {
        return false;
    }

    @Override
    public RecordReader<LongWritable, Text> createRecordReader(InputSplit split,
TaskAttemptContext context) {
        return new LineNumberRecordReader();
    }

    /** Wraps Hadoop's LineRecordReader and swaps the byte offset for a 1 based line counter. */
    public static class LineNumberRecordReader extends RecordReader<LongWritable, Text> {

        private final LineRecordReader lineReader = new LineRecordReader();
        private final LongWritable lineNumber = new LongWritable(0);

        @Override
        public void initialize(InputSplit split, TaskAttemptContext context) throws IOException {
            lineReader.initialize(split, context);
        }
```

```
        @Override
        public boolean nextKeyValue() throws IOException {
            if (!lineReader.nextKeyValue()) {
                return false;
            }
            lineNumber.set(lineNumber.get() + 1);
            return true;
        }

        @Override
        public LongWritable getCurrentKey() {
            return lineNumber;
        }

        @Override
        public Text getCurrentValue() {
            return lineReader.getCurrentValue();
        }

        @Override
        public float getProgress() throws IOException {
            return lineReader.getProgress();
        }

        @Override
        public void close() throws IOException {
            lineReader.close();
        }
    }
}
```

KeywordMatcher.java finds every case insensitive match on a line and returns the starting columns. The look behind and look ahead make it a whole word match, so `africa` matches "Africa" and "AFRICA" but not "African". Setting `search.wholeword=false` switches to partial matching.

```
package za.itbda.keywordsearch;

import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Finds every case insensitive occurrence of a keyword in a single line and
 * returns the 1 based column where each occurrence starts.
 *
 * In whole word mode "africa" matches "Africa" and "AFRICA" but not "African".
 * In partial mode it matches anywhere, so "African" is counted too.
 */
public class KeywordMatcher {

    private final Pattern pattern;

    public KeywordMatcher(String keyword, boolean wholeWord) {
        if (keyword == null || keyword.trim().isEmpty()) {
            throw new IllegalArgumentException("Keyword must not be empty");
        }
        String quoted = Pattern.quote(keyword.trim());
        String regex = wholeWord
                ? "(?<![\\p{L}\\p{N}])" + quoted + "(?![\\p{L}\\p{N}])"
                : quoted;
        this.pattern = Pattern.compile(regex, Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CASE);
    }

    public List<Integer> findColumns(String line) {
        List<Integer> columns = new ArrayList<>();
```

```
        Matcher m = pattern.matcher(line);
        while (m.find()) {
            columns.add(m.start() + 1);
        }
        return columns;
    }
}
```

KeywordSearchMapper.java:

```
package za.itbda.keywordsearch;

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.FileSplit;

/**
 * Map phase.
 *
 * Input  : (rowNumber, lineText) for one line of one document
 * Output : (documentName, "row,column") for every keyword hit on that line
 *
 * The keyword is read from the job configuration (search.keyword), so it is
 * never hardcoded. Because each document is one unsplit input, one mapper sees
 * a whole document and can report documents that have no match at all.
 */
public class KeywordSearchMapper extends Mapper<LongWritable, Text, Text, Text> {

    public static final String NO_MATCH = "NONE";

    public enum SearchCounters { LINES_SCANNED, KEYWORD_MATCHES, DOCUMENTS_WITH_MATCH }

    private final Text documentKey = new Text();
    private final Text location = new Text();
    private KeywordMatcher matcher;
    private long matchesInDocument = 0;

    @Override
    protected void setup(Context context) {
        Configuration conf = context.getConfiguration();
        String keyword = conf.get(KeywordSearchDriver.KEYWORD_PROPERTY);
        boolean wholeWord = conf.getBoolean(KeywordSearchDriver.WHOLE_WORD_PROPERTY, true);
        matcher = new KeywordMatcher(keyword, wholeWord);

        String fileName = ((FileSplit) context.getInputSplit()).getPath().getName();
        documentKey.set(fileName);
    }

    @Override
    protected void map(LongWritable rowNumber, Text line, Context context)
            throws IOException, InterruptedException {
        context.getCounter(SearchCounters.LINES_SCANNED).increment(1);

        for (int column : matcher.findColumns(line.toString())) {
            location.set(rowNumber.get() + "," + column);
            context.write(documentKey, location);
            matchesInDocument++;
            context.getCounter(SearchCounters.KEYWORD_MATCHES).increment(1);
        }
    }

    @Override
    protected void cleanup(Context context) throws IOException, InterruptedException {
```

```
        if (matchesInDocument == 0) {
            // Tell the reducer this document was searched but nothing was found
            location.set(NO_MATCH);
            context.write(documentKey, location);
        } else {
            context.getCounter(SearchCounters.DOCUMENTS_WITH_MATCH).increment(1);
        }
    }
}
```

Demonstration evidence

The job was run in two ways. First in Hadoop local mode, with the keyword typed in at the prompt to show it is not hardcoded:

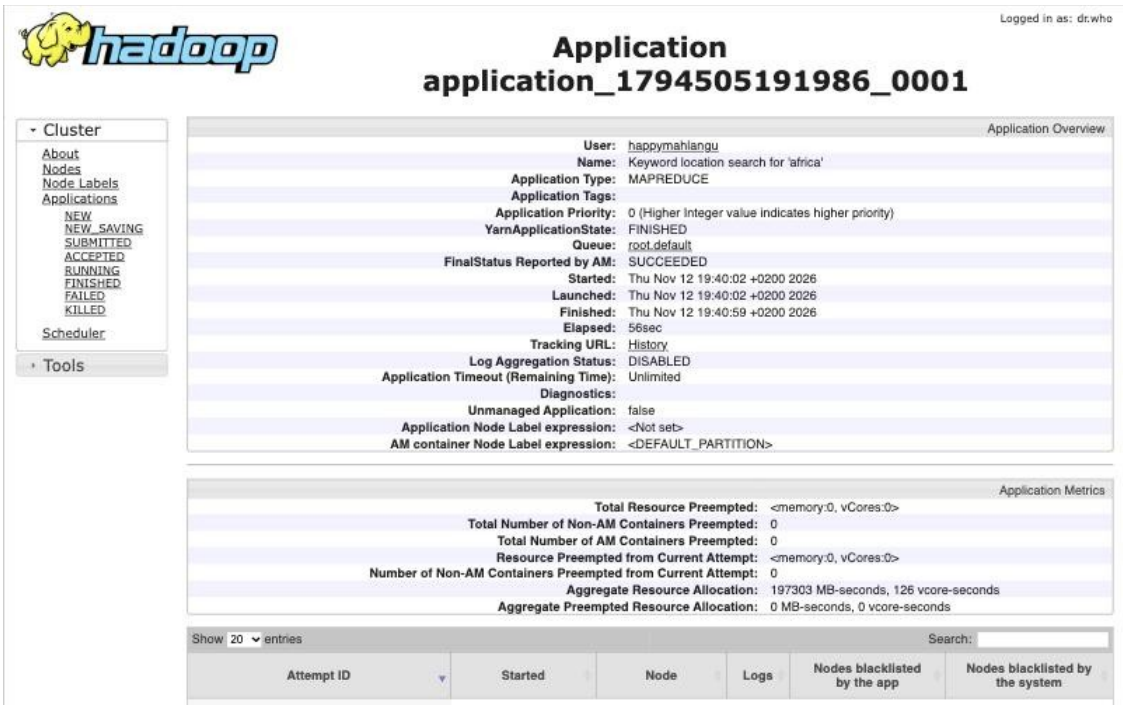
```
$ ./run_local.sh
Enter the keyword to search for: africa
Searching for keyword: africa (whole word = true, case insensitive)
...
Job job_local357785875_0001 completed successfully
    KEYWORD_MATCHES=16   DOCUMENTS_WITH_MATCH=4   LINES_SCANNED=307
```

Then on a single node cluster with HDFS and YARN (`./run_cluster.sh africa`). YARN launched 7 map tasks (one per document) and 1 reduce task:

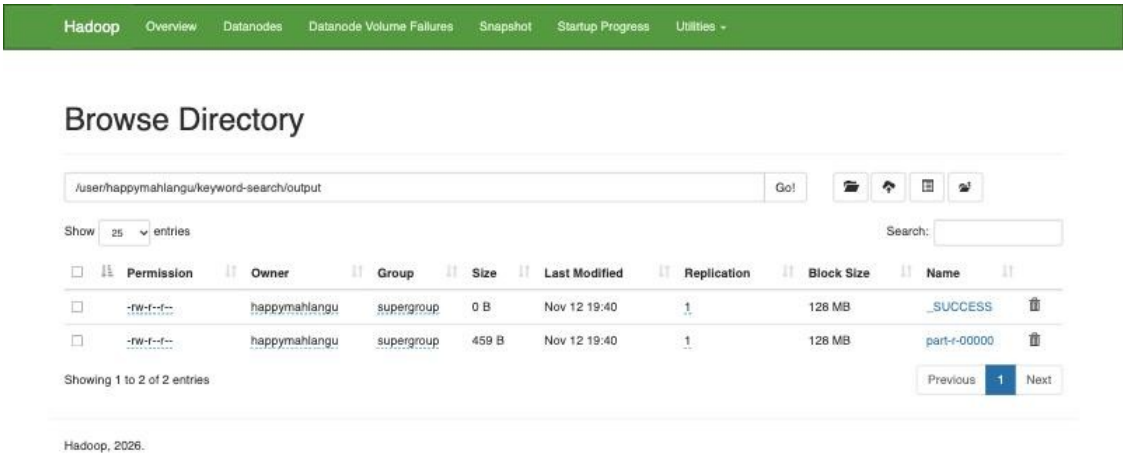
```
INFO mapreduce.Job: Running job: job_1794505191986_0001
INFO mapreduce.Job:  map 100% reduce 0%
INFO mapreduce.Job:  map 100% reduce 100%
INFO mapreduce.Job: Job job_1794505191986_0001 completed successfully
        Launched map tasks=7
        DOCUMENTS_WITH_MATCH=4
        KEYWORD_MATCHES=16
        LINES_SCANNED=307
```

Result read back from HDFS (`hdfs dfs -cat .../output/part-r-00000`):

```
doc1.txt    keyword=africa  matches=1   locations=(1,64)
doc2.txt    keyword=africa  matches=3   locations=(1,57) (33,24) (58,81)
doc3.txt    keyword=africa  matches=8   locations=(1,38) (3,7) (5,7) (40,146) (46,7) (50,142)
(52,86) (54,44)
doc4.txt    keyword=africa  matches=0   locations=not found
doc5.txt    keyword=africa  matches=4   locations=(1,62) (3,88) (23,121) (38,27)
doc6.txt    keyword=africa  matches=0   locations=not found
doc7.txt    keyword=africa  matches=0   locations=not found
```



YARN ResourceManager showing the job finished with status SUCCEEDED



HDFS file browser showing the job output folder

To check the result independently, a separate Python script scanned the same files with the same rule. It found exactly the same 16 positions. Running with `-D search.keyword=AFRICA -D search.wholeword=false` (upper case input, partial matching) returned 22 matches, because words such as "African" are then counted too. That confirms case insensitivity and the switch both work. The full logs are in `q1-mapreduce/evidence/`.

1.3 Reducer implementation

```
package za.itbda.keywordsearch;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.StringJoiner;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

/**
 * Reduce phase.
 *
 * Input  : (documentName, ["row,column", "row,column", ...])
 * Output : documentName  TAB  keyword=x  matches=n  locations=(r,c) (r,c) ...
 *
 * The shuffle groups every location of a document under one key. The reducer
 * sorts them by row then column so the result reads top to bottom like the
 * original document.
 */
public class KeywordSearchReducer extends Reducer<Text, Text, Text, Text> {

    private final Text result = new Text();
    private String keyword;

    @Override
    protected void setup(Context context) {
        keyword = context.getConfiguration().get(KeywordSearchDriver.KEYWORD_PROPERTY);
    }

    @Override
    protected void reduce(Text document, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
        List<long[]> locations = new ArrayList<>();

        for (Text value : values) {
            String v = value.toString();
            if (KeywordSearchMapper.NO_MATCH.equals(v)) {
                continue;
            }
            String[] parts = v.split(",");
            locations.add(new long[] { Long.parseLong(parts[0]), Long.parseLong(parts[1]) });
        }

        locations.sort((a, b) -> a[0] != b[0] ? Long.compare(a[0], b[0]) : Long.compare(a[1],
b[1]));

        StringJoiner joined = new StringJoiner(" ");
        for (long[] loc : locations) {
            joined.add("(" + loc[0] + "," + loc[1] + ")");
        }

        String locationText = locations.isEmpty() ? "not found" : joined.toString();
        result.set("keyword=" + keyword + "\tmatches=" + locations.size() + "\tlocations=" +
locationText);
        context.write(document, result);
    }
}
```

The reducer is called once per document. Hadoop has already grouped every `"row,column"` value for that document during the shuffle. The reducer parses them, ignores the `NONE` marker, sorts them so they read top to bottom like the original text, and writes the keyword, the number of matches and the ordered `(row,col)` list. A document whose only value was `NONE` is reported as `matches=0 locations=not found`.

1.4 Hadoop MapReduce configuration

Configuration happens at two levels: the job configuration in the driver, and the cluster configuration in the Hadoop XML files.

Job configuration (KeywordSearchDriver.java):

```
package za.itbda.keywordsearch;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

/**
 * Job configuration and entry point.
 *
 * Usage:
 *   hadoop jar keyword-search.jar input_dir output_dir [keyword]
 *
 * The keyword is supplied at run time in one of three ways, checked in order:
 *   1. third command line argument
 *   2. generic option   -D search.keyword=africa
 *   3. typed in at the prompt when neither of the above is given
 *
 * Optional switches:
 *   -D search.wholeword=false   also match inside longer words, for example African
 *   -D search.overwrite=true    delete an existing output folder before running
 */
public class KeywordSearchDriver extends Configured implements Tool {

    public static final String KEYWORD_PROPERTY = "search.keyword";
    public static final String WHOLE_WORD_PROPERTY = "search.wholeword";
    public static final String OVERWRITE_PROPERTY = "search.overwrite";

    @Override
    public int run(String[] args) throws Exception {
        if (args.length < 2) {
            System.err.println("Usage: hadoop jar keyword-search.jar <input dir> <output dir>
[keyword]");
            return 2;
        }

        Configuration conf = getConf();
        String keyword = resolveKeyword(args, conf);
        if (keyword == null || keyword.isEmpty()) {
            System.err.println("No keyword supplied, nothing to search for.");
            return 2;
        }
        conf.set(KEYWORD_PROPERTY, keyword);

        Path input = new Path(args[0]);
        Path output = new Path(args[1]);
        FileSystem fs = output.getFileSystem(conf);
        if (fs.exists(output) && conf.getBoolean(OVERWRITE_PROPERTY, false)) {
            fs.delete(output, true);
```

```
        }

        Job job = Job.getInstance(conf, "Keyword location search for '" + keyword + "'");
        job.setJarByClass(KeywordSearchDriver.class);

        job.setInputFormatClass(LineNumberInputFormat.class);
        job.setMapperClass(KeywordSearchMapper.class);
        job.setReducerClass(KeywordSearchReducer.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        // One reducer gives a single result file sorted by document name
        job.setNumReduceTasks(1);

        FileInputFormat.addInputPath(job, input);
        FileOutputFormat.setOutputPath(job, output);

        System.out.println("Searching for keyword: " + keyword
                + " (whole word = " + conf.getBoolean(WHOLE_WORD_PROPERTY, true) + ", case
insensitive)");
        return job.waitForCompletion(true) ? 0 : 1;
    }

    private static String resolveKeyword(String[] args, Configuration conf) throws Exception {
        if (args.length >= 3) {
            return args[2].trim();
        }
        String fromConf = conf.get(KEYWORD_PROPERTY);
        if (fromConf != null && !fromConf.trim().isEmpty()) {
            return fromConf.trim();
        }
        System.out.print("Enter the keyword to search for: ");
        System.out.flush();
        BufferedReader in = new BufferedReader(new InputStreamReader(System.in,
StandardCharsets.UTF_8));
        String line = in.readLine();
        return line == null ? null : line.trim();
    }

    public static void main(String[] args) throws Exception {
        System.exit(ToolRunner.run(new Configuration(), new KeywordSearchDriver(), args));
    }
}
```

Cluster configuration (`q1-mapreduce/hadoop-config/`, used through `HADOOP_CONF_DIR`):

File	Setting	Value	Purpose
core-site.xml	`fs.defaultFS`	`hdfs://localhost:9000`	Address of the HDFS NameNode
core-site.xml	`hadoop.tmp.dir`	`/tmp/hadoop-${user.name}`	Base folder for Hadoop data
hdfs-site.xml	`dfs.replication`	`1`	One copy of each block on a single node
hdfs-site.xml	`dfs.blocksize`	`134217728`	128 MB HDFS blocks
mapred-site.xml	`mapreduce.framework.name`	`yarn`	Run MapReduce jobs on YARN

File	Setting	Value	Purpose
mapred-site.xml	`mapreduce.map.memory.mb`, `mapreduce.reduce.memory.mb`	`1024`	Container size for each task
mapred-site.xml	`mapreduce.job.reduces`	`1`	Single sorted result file
yarn-site.xml	`yarn.resourcemanager.hostname`	`localhost`	Where jobs are submitted
yarn-site.xml	`yarn.nodemanager.aux-services`	`mapreduce_shuffle`	Shuffle service reducers pull map output from
yarn-site.xml	`yarn.nodemanager.resource.memory-mb`	`4096`	Memory YARN can hand out on this node

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- MapReduce settings: run jobs on YARN and size the map and reduce containers -->
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
    <property>
        <name>mapreduce.application.classpath</name>
        <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:$HADOOP_MAPRED_HOME/share/hadoop/
mapreduce/lib/*</value>
    </property>
    <property>
        <name>yarn.app.mapreduce.am.env</name>
        <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
    </property>
    <property>
        <name>mapreduce.map.env</name>
        <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
    </property>
    <property>
        <name>mapreduce.reduce.env</name>
        <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
    </property>
    <property>
        <name>mapreduce.map.memory.mb</name>
        <value>1024</value>
    </property>
    <property>
        <name>mapreduce.reduce.memory.mb</name>
        <value>1024</value>
    </property>
    <property>
        <name>mapreduce.job.reduces</name>
        <value>1</value>
    </property>
</configuration>

<?xml version="1.0" encoding="UTF-8"?>
<!-- YARN settings: resource manager location and the shuffle service reducers pull from -->
<configuration>
    <property>
        <name>yarn.resourcemanager.hostname</name>
        <value>localhost</value>
    </property>
    <property>
        <name>yarn.nodemanager.hostname</name>
        <value>localhost</value>
    </property>
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
    <property>
```

```
        <name>yarn.nodemanager.env-whitelist</name>

<value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,
HADOOP_YARN_HOME,HADOOP_HOME,PATH,LANG,TZ,HADOOP_MAPRED_HOME</value>
    </property>
    <property>
        <name>yarn.nodemanager.resource.memory-mb</name>
        <value>4096</value>
    </property>
    <property>
        <name>yarn.nodemanager.vmem-check-enabled</name>
        <value>false</value>
    </property>
    <!-- Dev laptop setting: the default marks the node unhealthy once the disk is 90% full -->
    <property>
        <name>yarn.nodemanager.disk-health-checker.max-disk-utilization-per-disk-percentage</
name>
        <value>99.0</value>
    </property>
</configuration>
```

The run script starts the daemons with `hdfs --daemon start namenode`, `hdfs --daemon start datanode`, `yarn --daemon start resourcemanager` and `yarn --daemon start nodemanager`, so no SSH setup is needed. It then uploads the input with `hdfs dfs -put` and submits the job with `hadoop jar target/keyword-search.jar <input> <output> [keyword]`.

Question 2: Topic modelling with LDA

The full notebook with all code and output is `q2-lda/q2_lda_topic_modelling.Rmd` (rendered as `.html`). The number of topics and the number of words per topic are notebook parameters (`n_topics = 4`, `top_k = 10`), so they can be changed without touching the code.

2.1 Data frame and denoising

Every file in `news.zip` becomes one row of a data frame with two columns, `file_name` and `content`:

```
news_dir <- "../data/extracted/sa_news"
if (!dir.exists(news_dir)) {
  unzip("../data/raw/news.zip", exdir = "../data/extracted")
}

files <- list.files(news_dir, pattern = "\\.txt$", full.names = TRUE)
files <- files[order(as.integer(str_extract(basename(files), "\\d+")))]

news_df <- data.frame(
  file_name = basename(files),
  content   = vapply(files, function(f) paste(readLines(f, encoding = "UTF-8", warn = FALSE),
                                              collapse = "\n"), character(1)),
  stringsAsFactors = FALSE,
  row.names = NULL
)

str(news_df)
news_df %>%
  mutate(content = paste0(str_sub(str_squish(content), 1, 110), "...")) %>%
  knitr::kable(caption = "Raw data frame, first 110 characters of each article")
```

The ten articles are South African government news items from July 2024. Profiling the raw text showed the noise to remove: every file starts with a dateline (for example "Tuesday, July 9, 2024"), there are curly quotes and other non ASCII characters, money amounts such as "R30 000", and blank or padded lines. The cleaning pipeline:

Step	Technique	Why
1	Normalise non breaking spaces, curly quotes and dashes	The same word should not appear in several encodings
2	Remove the dateline	Dates repeat in every file and say nothing about the topic
3	Lower case, and join "South Africa" into one token	"Eskom" and "eskom" become one term, and the country name is not split
4	Remove URLs, numbers, currency amounts and punctuation	They add sparse, meaningless terms
5	Remove English and SMART stop words plus news filler (said, says, also, will, month names)	These words appear everywhere and hide the topic words
6	Lemmatise	"governments" and "government" become the same term
7	Drop tokens shorter than 3 letters	Removes leftovers from possessives

```
custom_stop <- c("said", "says", "say", "also", "will", "can", "one", "two", "three", "make",
                 "mr", "ms", "per", "cent", "including", "according", "however",
                 "monday", "tuesday", "wednesday", "thursday", "friday", "saturday",
                 "sunday", "january", "february", "march", "april", "may", "june",
                 "july", "august", "september", "october", "november", "december")
```

```
all_stop <- unique(c(stopwords("en"), stopwords("SMART"), custom_stop))

clean_text <- function(x) {
  x %>%
    str_replace_all("\r\n?", "\n") %>%                                    # 1
    str_replace_all("[\\u00A0\\u2009\\u202F]", " ") %>%
    str_replace_all("[\\u2018\\u2019]", "'") %>%
    str_replace_all("[\\u201C\\u201D]", "\"") %>%
    str_replace_all("[\\u2010-\\u2015]", " ") %>%
    str_remove_all("(Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday), \\w+ \\d{1,2}, \
\d{4}") %>%  # 2
    str_to_lower() %>%                                                    # 3
    str_replace_all("south africa(n|ns)?", "south_africa") %>%
    str_remove_all("https?://\\S+|www\\.\\S+") %>%                       # 4
    str_replace_all("'s\\b", " ") %>%
    str_replace_all("[^a-z_\\s]", " ") %>%
    str_squish()
}

tokens <- news_df %>%
  mutate(clean = clean_text(content)) %>%
  select(file_name, clean) %>%
  unnest_tokens(word, clean) %>%
  filter(!word %in% all_stop) %>%                                         # 5
  mutate(word = lemmatize_words(word)) %>%                                # 6
  filter(!word %in% all_stop, nchar(word) >= 3)                           # 7

news_df <- news_df %>%
  left_join(tokens %>% group_by(file_name) %>%
              summarise(clean_content = paste(word, collapse = " ")),
            by = "file_name")

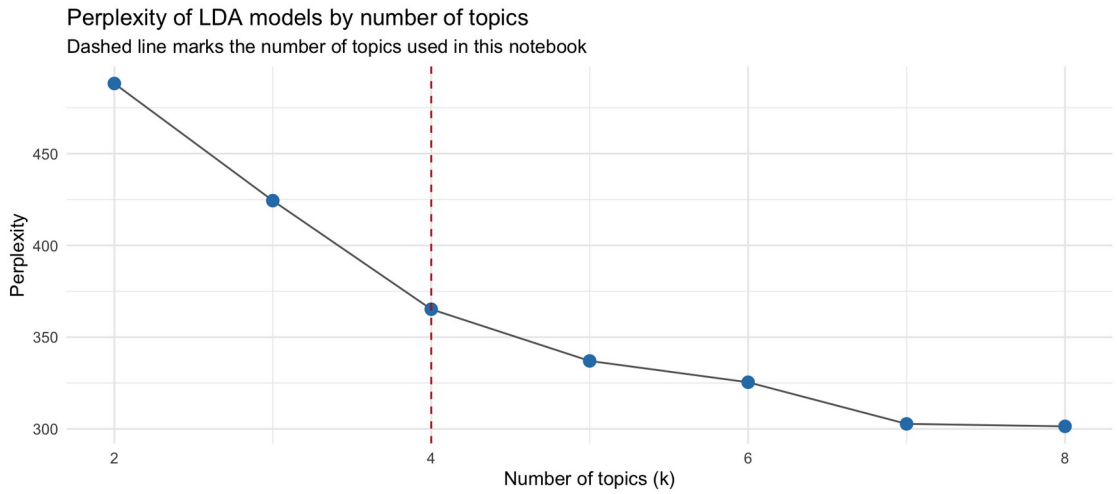
news_df %>%
  transmute(file_name,
            raw_words   = str_count(content, "\\S+"),
            clean_words = str_count(clean_content, "\\S+"),
            removed_pct = round(100 * (1 - clean_words / raw_words), 1),
            clean_preview = paste0(str_sub(clean_content, 1, 80), "..."))

dir.create("../data/processed", showWarnings = FALSE)
write_csv(news_df, "../data/processed/news_clean.csv")
```

Cleaning removed roughly half of every article (between 46% and 54% of the words), leaving 875 distinct terms across the 10 documents. The cleaned data frame is saved as `data/processed/news_clean.csv`.

2.2 LDA topics and word distributions

A document term matrix was built from the cleaned tokens. To choose the number of topics, models with 2 to 8 topics were compared on perplexity. Perplexity always falls as more topics are added, so the elbow of the curve was used: the big gains stop at k = 4.



Perplexity by number of topics

The model was fitted with Gibbs sampling (2,000 iterations, alpha = 0.1, fixed seed for repeatable results):

```
lda_model <- LDA(dtm, k = params$n_topics, method = "Gibbs",
                 control = list(seed = 2026, iter = 2000, burnin = 500, alpha = 0.1))

beta  <- tidy(lda_model, matrix = "beta")    # word distribution per topic
gamma <- tidy(lda_model, matrix = "gamma")   # topic distribution per document
```

Top topics ranked by prevalence (average share across all documents):

Rank	Topic	Prevalence	Theme
1	4	0.313	Service delivery and financial accountability
2	3	0.287	Public safety and weather alerts
3	2	0.216	Education and labour priorities
4	1	0.184	Coalition government and public administration

Top 10 words per topic with their probability (beta):

Rank	Topic 1	Topic 2	Topic 3	Topic 4
1	government (0.042)	department (0.037)	police (0.022)	service (0.033)
2	public (0.033)	priority (0.018)	cape (0.016)	invoice (0.030)
3	coalition (0.030)	school (0.014)	cold (0.016)	department (0.021)
4	administration (0.021)	unemployment (0.014)	expect (0.014)	pay (0.021)
5	country (0.019)	continue (0.012)	weather (0.014)	eskom (0.020)
6	minister (0.014)	meth (0.012)	west (0.014)	day (0.020)
7	role (0.014)	ensure (0.011)	western (0.014)	psc (0.020)
8	governance (0.011)	report (0.011)	team (0.012)	gxoyiya (0.016)
9	policy (0.011)	labour (0.011)	sandf (0.012)	provider (0.015)

Rank	Topic 1	Topic 2	Topic 3	Topic 4
10	relationship (0.011)	work (0.009)	week (0.010)	official (0.013)
Top 10 share	0.204	0.148	0.142	0.209

Topic distribution per document (gamma):

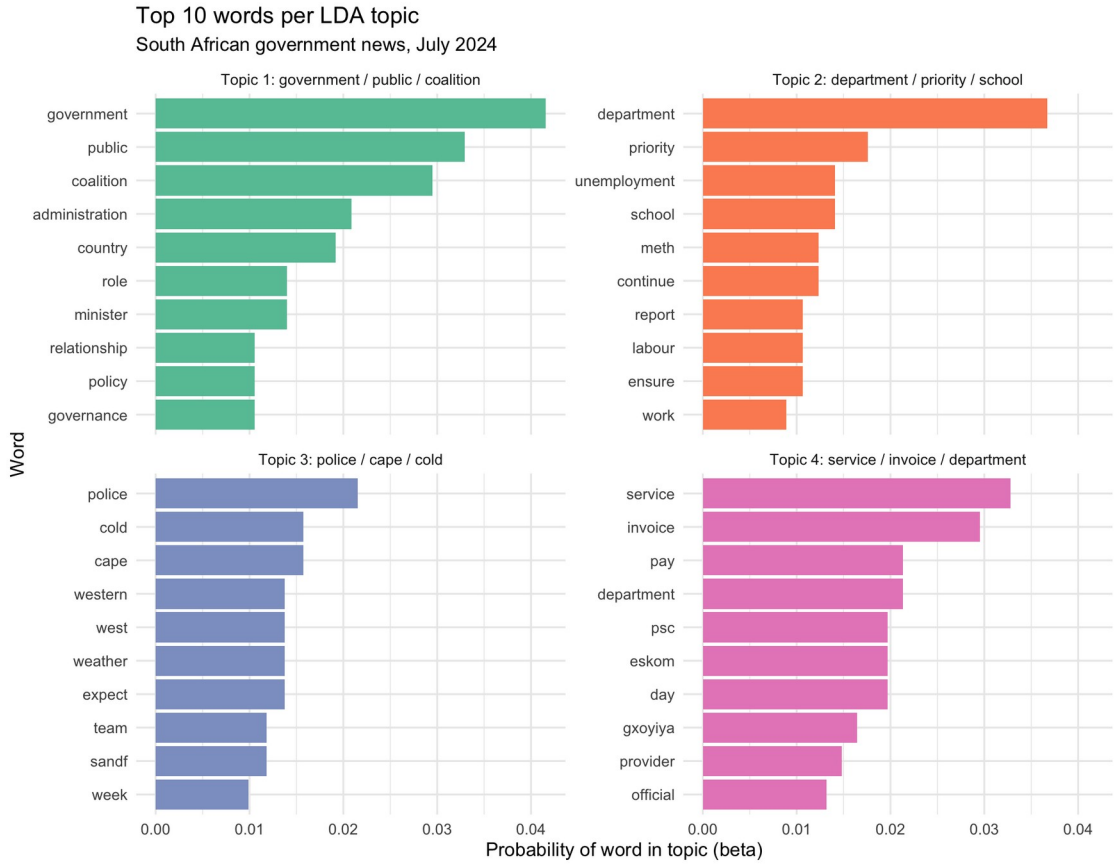
Document	T1	T2	T3	T4	Dominant	Headline
news_7	0.630	0.070	0.001	0.300	1	Democratic dispensation necessitated a complete overhaul of state bureaucracy
news_8	0.917	0.083	0.000	0.000	1	Trust and relationships key ingredients to coalition governments
news_2	0.001	0.998	0.001	0.001	2	Gauteng Education commits to implementing Mayibuye Primary School remedial actions
news_3	0.171	0.828	0.000	0.000	2	New Labour Minister charts the way forward
news_10	0.000	0.092	0.829	0.079	3	Wet, cold weather expected to continue in Western Cape
news_4	0.001	0.001	0.988	0.010	3	Police recue two kidnapped victims in Benoni
news_5	0.001	0.048	0.950	0.001	3	President Ramaphosa sends condolences to deceased SANDF members
news_1	0.001	0.034	0.101	0.863	4	Eskom former official, service provider appear in court
news_6	0.000	0.000	0.000	0.999	4	Accounting officers must be accountable for non-payment of service providers
news_9	0.118	0.001	0.001	0.881	4	PSC applauds officials who refuse to accept a bribes

Interpretation. Nine of the ten articles belong mostly to one topic (gamma above 0.8), so the four topics separate the news well. Topic 4 is the most prevalent and groups the three stories about how public money is handled: the Eskom fraud case, late payment of suppliers and the Public Service Commission on bribes. Topic 3 brings together safety and emergency stories (police, SANDF, Western Cape weather). Topic 2 covers education and labour, and Topic 1 covers coalition government and the public service. The clearest mixed article is news_7, which is 63% Topic 1 and 30% Topic 4 because it discusses bureaucracy reform and accountability together. The beta values are small (the strongest words sit between 0.02 and 0.04) because 875 terms are shared among only 10 short documents, so the ranking of words matters more than their size.

2.3 Visualising the top words per topic

Three methods were used.

Method 1: faceted bar chart of the top 10 words and their beta per topic. It shows the exact weights and makes the ranking easy to read.



Top 10 words per LDA topic

Method 2: word clouds of the top 30 words per topic, with word size proportional to beta. They give a quick sense of each theme at a glance.



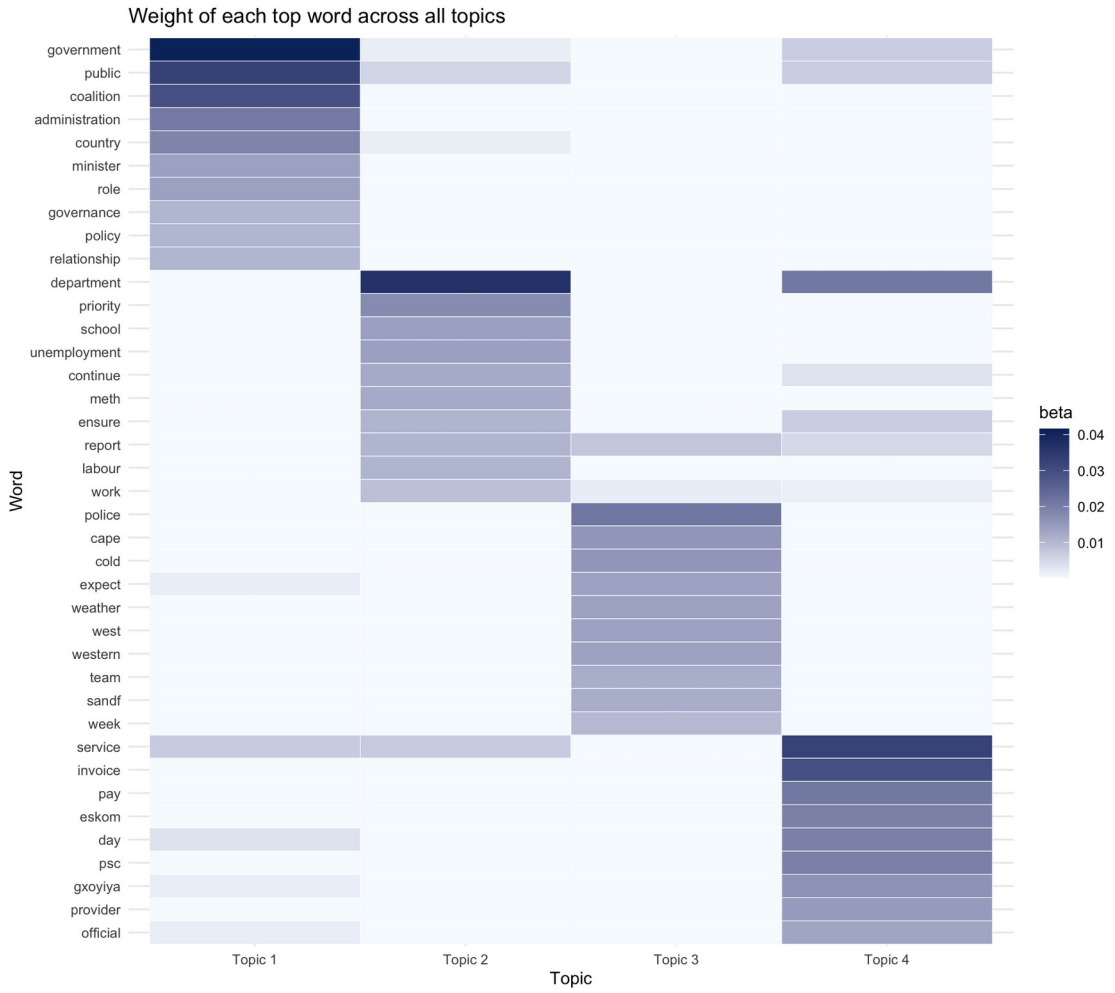






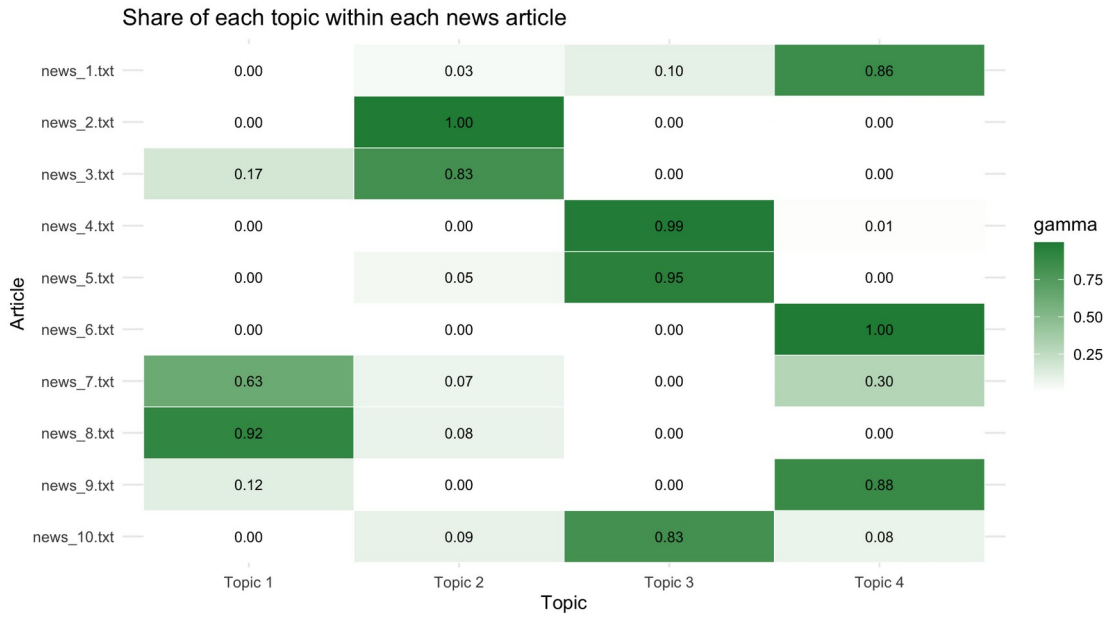
Word cloud per topic

Method 3: heat map of every top word against every topic. It shows that the topics overlap very little. The clearest shared word is "department", which is strong in both Topic 2 (education department) and Topic 4 (departments paying invoices), while "government", "public" and "service" have small weights outside their main topic.



Heat map of word weights across topics

The document to topic heat map confirms the gamma table above.



Share of each topic within each article

Question 3: WebText API proof of concept

The notebook is `q3-webtext/q3_webtext_mining.Rmd`. Its only input is a web link (`params$url`). The page chosen is the Wikipedia article **Climate change in Africa** (https://en.wikipedia.org/wiki/Climate_change_in_Africa), which fits the brief's theme of language as environmental data. At the end of the notebook the whole pipeline is wrapped in one function, `webtext_api(url)`, which is the shape a real API endpoint would take.

3.1 Extracting clean web text into an R data structure

Fair use of web scraping. Before scraping, the code checks `robots.txt` and stops if the path is not allowed. Only one page is requested, with a clear user agent and a delay, and the HTML is cached in `data/raw/` so knitting the notebook again does not hit the site. Wikipedia text is licensed CC BY-SA 4.0, which allows reuse with attribution, and the licence and source URL are stored with the data. No personal data is collected.

```
check_scraping_allowed <- function(url) {
  allowed <- robotstxt::paths_allowed(url, warn = FALSE)
  if (!isTRUE(allowed)) stop("robots.txt does not allow scraping: ", url)
  allowed
}

check_scraping_allowed(params$url)

fetch_page <- function(url, cache_dir = "../data/raw", refresh = FALSE) {
  cache_file <- file.path(cache_dir, paste0("q3_", str_replace_all(basename(url), "[^A-Za-z0-9]",
"_"), ".html"))
  if (file.exists(cache_file) && !refresh) {
    return(list(html = read_html(cache_file, encoding = "UTF-8"), cache_file = cache_file,
from_cache = TRUE))
  }
  Sys.sleep(1)  # be polite even for a single request
  resp <- GET(url, user_agent("itbda-student-webtext-poc/1.0 (educational text mining project)"))
  stop_for_status(resp)
  html_text <- content(resp, as = "text", encoding = "UTF-8")
  dir.create(cache_dir, showWarnings = FALSE, recursive = TRUE)
  writeLines(html_text, cache_file, useBytes = TRUE)
  list(html = read_html(html_text), cache_file = cache_file, from_cache = FALSE)
}

page <- fetch_page(params$url, refresh = params$refresh)
page[c("cache_file", "from_cache")]
```

Headings and paragraphs are read in page order so that every paragraph keeps its section. Infoboxes, tables, captions and navigation are left out because only `h2`, `h3` and `p` nodes are selected.

```
extract_article <- function(html) {
  body  <- html_element(html, "#mw-content-text .mw-parser-output")
  nodes <- html_elements(body, xpath = ".//h2 | .//h3 | .//p")

  section <- "Introduction"; subsection <- NA_character_
  rows <- vector("list", length(nodes))
  for (i in seq_along(nodes)) {
    tag  <- html_name(nodes[[i]])
    text <- html_text2(nodes[[i]])
    if (tag == "h2") { section <- text; subsection <- NA_character_; next }
    if (tag == "h3") { subsection <- text; next }
    rows[[i]] <- tibble(section = section, subsection = subsection, raw_text = text)
  }
  bind_rows(rows) %>% mutate(para_id = row_number(), .before = 1)
}
```

```
webtext <- list(
  meta = list(
    url           = params$url,
    title         = html_text2(html_element(page$html, "#firstHeading")),
    last_modified = html_text2(html_element(page$html, "#footer-info-lastmod")),
    retrieved     = format(file.info(page$cache_file)$mtime, "%Y-%m-%d %H:%M"),
    licence       = "CC BY-SA 4.0, Wikipedia contributors"
  ),
  paragraphs = extract_article(page$html)
)

str(webtext, max.level = 2)
webtext$paragraphs %>% count(section, name = "paragraphs")
webtext$paragraphs %>% slice(1:4) %>% mutate(raw_text = str_trunc(raw_text, 160))
```

The result is a named list. `meta` holds the URL, title, last edit date, retrieval time and licence. `paragraphs` is a data frame with 101 rows (`para_id`, `section`, `subsection`, `raw_text`) spread across 8 sections, with Adaptation (39 paragraphs), Socioeconomic impacts (17) and Impacts by region (17) the largest.

3.2 Preprocessing

Profiling the raw paragraphs found 266 Wikipedia citation markers such as `[96]`, 53 non ASCII characters (curly quotes, en dashes, degree signs) and one empty paragraph. Cleaning is done in two layers. Sentences stay readable, because extractive summaries reuse real sentences. Words are reduced to lemmas for counting.

Step	Technique	Purpose
1	Drop non content sections (See also, References, External links)	Removes link lists and bibliography
2	Strip citation markers such as [12], [a] and [citation needed]	Footnotes are not part of the prose
3	Normalise quotes, dashes and non breaking spaces, collapse whitespace	One consistent character set
4	Remove very short paragraphs and duplicates	Captions and stubs add little
5	Split into sentences, drop fragments under 6 words	Units for summarisation
6	Lower case, remove numbers, punctuation and stop words, plus title words (climate, change, africa)	Keeps meaningful content words
7	Lemmatise	"droughts" and "drought" count once

```
drop_sections <- c("See also", "References", "External links", "Further reading", "Notes")

clean_sentence_text <- function(x) {
  x %>%
    str_remove_all("\\[[^\\]]{1,30}\\]") %>%                    # 2
    str_replace_all("[\\u2018\\u2019]", "'") %>%                  # 3
    str_replace_all("[\\u201C\\u201D]", "\"") %>%
    str_replace_all("[\\u2013\\u2014]", " to ") %>%
    str_replace_all("[\\u00A0\\u2009\\u202F]", " ") %>%
    str_replace_all("\\s+([,.;:])", "\\1") %>%
    str_squish()
}

paras <- webtext$paragraphs %>%
  filter(!section %in% drop_sections) %>%                       # 1
  mutate(text = clean_sentence_text(raw_text)) %>%
```

```
  filter(str_count(text, "\\S+") >= 12) %>%                          # 4
  distinct(text, .keep_all = TRUE)

sentences <- paras %>%
  select(para_id, section, text) %>%
  unnest_tokens(sentence, text, token = "sentences", to_lower = FALSE) %>%   # 5
  filter(str_count(sentence, "\\S+") >= 6) %>%
  mutate(sentence_id = row_number(), .before = 1)

boilerplate <- c("climate", "change", "africa", "african", "wikipedia", "also", "can", "may",
                 "will", "however", "including", "et", "al")

words <- sentences %>%
  select(sentence_id, section, sentence) %>%
  unnest_tokens(word, sentence) %>%                                  # 6
  filter(!str_detect(word, "\\d"), !word %in% stop_words$word) %>%
  mutate(lemma = lemmatize_words(word)) %>%                          # 7
  filter(!lemma %in% c(stop_words$word, boilerplate), nchar(lemma) > 2)

tibble(
  stage = c("Raw paragraphs", "Clean paragraphs", "Sentences", "Content words (lemmas)", "Unique
lemmas"),
  count = c(nrow(webtext$paragraphs), nrow(paras), nrow(sentences), nrow(words),
n_distinct(words$lemma))
)

paras %>% slice_max(str_count(raw_text, "\\["), n = 1, with_ties = FALSE) %>% select(raw_text,
text) %>% t() %>% as.data.frame() %>%
  setNames("Before and after cleaning") %>% mutate(across(everything(), ~ str_trunc(.x, 400)))

dir.create("../data/processed", showWarnings = FALSE)
write_csv(paras %>% select(para_id, section, subsection, text),
"../data/processed/q3_paragraphs_clean.csv")
```

Stage	Count
Raw paragraphs	101
Clean paragraphs	100
Sentences	322
Content words (lemmas)	3,843
Unique lemmas	1,231

Example before and after: "...a general decrease in March – May (MAM) seasonal rains with a slight increase during June – September (JJAS) and October – December (OND) rains,[96] although there appears to have been a recent recovery in the MAM rains.[97]" becomes "...a general decrease in March to May (MAM) seasonal rains with a slight increase during June to September (JJAS) and October to December (OND) rains, although there appears to have been a recent recovery in the MAM rains."

3.3 Text summarisation

Three summarisation techniques were applied, each reducing the 322 sentences to 6.

Technique 1: word frequency scoring (Luhn method)

Each sentence is scored by the normalised frequency of its content words, divided by the square root of its length so that long sentences are not automatically favoured.

```
word_freq <- words %>% count(lemma, sort = TRUE) %>% mutate(weight = n / max(n))

freq_scores <- words %>%
```

```
  left_join(word_freq, by = "lemma") %>%
  group_by(sentence_id) %>%
  summarise(freq_score = sum(weight) / sqrt(n()), .groups = "drop")   # sqrt damps the bias to
long sentences

freq_summary <- sentences %>%
  inner_join(freq_scores, by = "sentence_id") %>%
  slice_max(freq_score, n = params$n_sentences) %>%
  arrange(sentence_id)

freq_summary %>% select(sentence_id, section, sentence)
```

Summary: (1) The impacts include higher temperatures, droughts, changing rainfall patterns, and increased climate variability. (2) Others refer to this as a period of "hydrological intensification" with much of the annual rainfall increase coming from more severe rain events and sometimes flooding... (3) The World Health Organization's report "Adaptation to Climate Change in Africa Plan of Action for the Health Sector 2012 to 2016" is intended to "provide a comprehensive and evidence-based coordinated response..." (4) Key adaptations in northern Africa relate to increased risk of water scarcity... (5) In Eastern Africa adaptation options are varied, including improving use of climate information, actions in the agriculture and livestock sector, and in the water sector. (6) South Africa has adopted in August 2020 its National Climate Change Adaptation Strategy...

Technique 2: TextRank

Sentences are nodes in a graph, linked by how many lemmas they share. PageRank picks the sentences most connected to the rest of the article.

```
tr <- textrank_sentences(
  data        = sentences %>% transmute(textrank_id = sentence_id, sentence),
  terminology = words %>% transmute(textrank_id = sentence_id, lemma)
)

textrank_summary <- tr$sentences %>%
  rename(sentence_id = textrank_id, textrank = textrank) %>%
  slice_max(textrank, n = params$n_sentences) %>%
  arrange(sentence_id) %>%
  left_join(sentences %>% select(sentence_id, section), by = "sentence_id")

textrank_summary %>% select(sentence_id, section, sentence)
```

Summary: (1) It is estimated that by 2030, the people of Africa will be exposed to a rise in sea level due to an increase in temperature. (2) The impacts include higher temperatures, droughts, changing rainfall patterns, and increased climate variability. (3) The agriculture sector is sensitive to climate variability, especially the inter-annual variability of precipitation, temperature patterns, and extreme weather events (droughts and floods). (4) Climate change in Kenya is expected to have large impacts on the agricultural sector, which is predominantly rain-fed... (5) Reduced water availability, in turn, interacts with increasing temperatures to create need for adaptation among rainfed wheat production and changing disease risk... (6) In Eastern Africa adaptation options are varied...

Technique 3: LexRank

Also graph based, but sentences are compared with TF-IDF weighted cosine similarity, so shared rare words count for more than shared common words.

```
lex <- lexRank(sentences$sentence, docId = sentences$sentence_id,
               n = params$n_sentences, continuous = TRUE, returnTies = FALSE,
               usePageRank = TRUE, removePunc = TRUE, removeNum = TRUE, toLower = TRUE,
               stemWords = TRUE, rmStopWords = TRUE, Verbose = FALSE)

lexrank_summary <- lex %>%
  mutate(sentence_id = as.integer(docId)) %>%
```

```
  arrange(sentence_id) %>%
  left_join(sentences %>% select(sentence_id, section), by = "sentence_id")

lexrank_summary %>% select(sentence_id, section, sentence)
```

Summary: (1) Climate change in Africa is a serious threat because the continent is one of the most vulnerable regions to the effects of climate change, despite being responsible for less than 4% of global greenhouse gas emissions. (2) The impacts include higher temperatures, droughts, changing rainfall patterns, and increased climate variability. (3) Africa will be one of the regions most impacted by the adverse effects of climate change. (4) The impacts of climate change on agricultural production in Africa will have serious implications for food security and livelihoods. (5) Climate change will significantly affect agriculture in West Africa by increasing the variability in food production, access and availability. (6) In the future, both rainfall and temperature are projected to change over Eastern Africa.

Comparing the methods

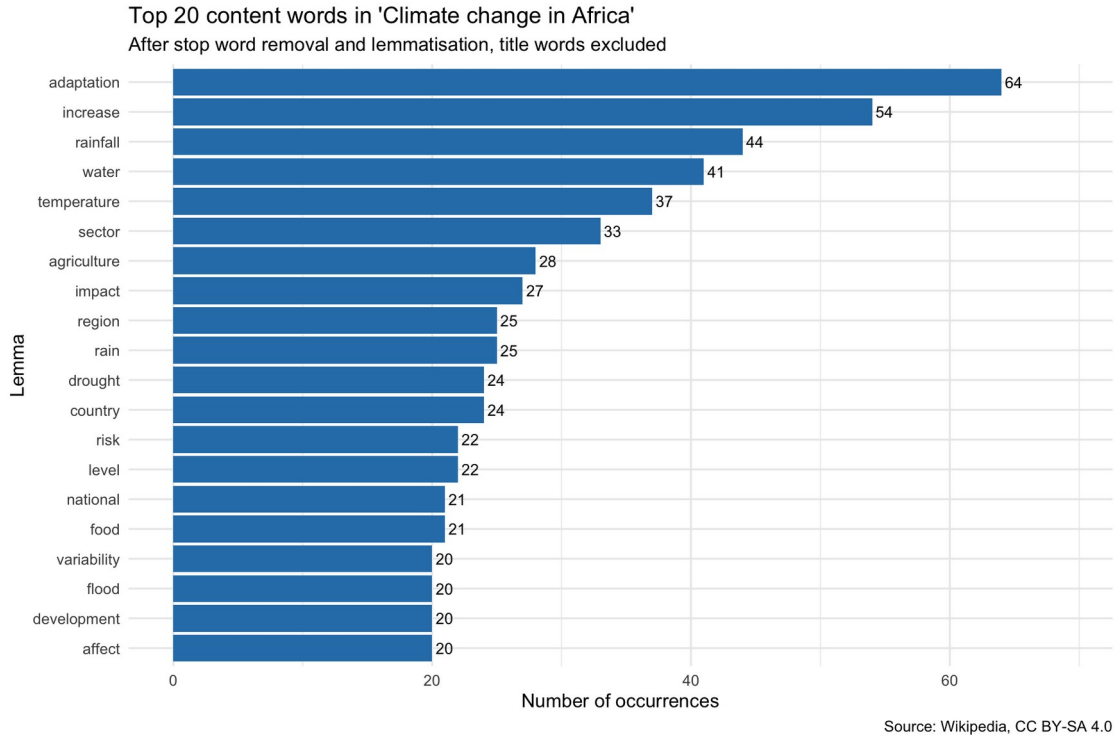
Method	Sentences	Words kept	Compression	Sections covered
Frequency	6	222	2.8%	3
TextRank	6	145	1.8%	3
LexRank	6	117	1.5%	3

The Jaccard overlap between the chosen sentences is low (0.20 between Frequency and TextRank, 0.09 for both pairs involving LexRank). All three agree on only sentence 65, "The impacts include higher temperatures, droughts, changing rainfall patterns, and increased climate variability", which is therefore the best single line summary of the page. Because each six sentence summary only touches three of the eight sections, a structured summary was also built from the highest TextRank sentence in each section:

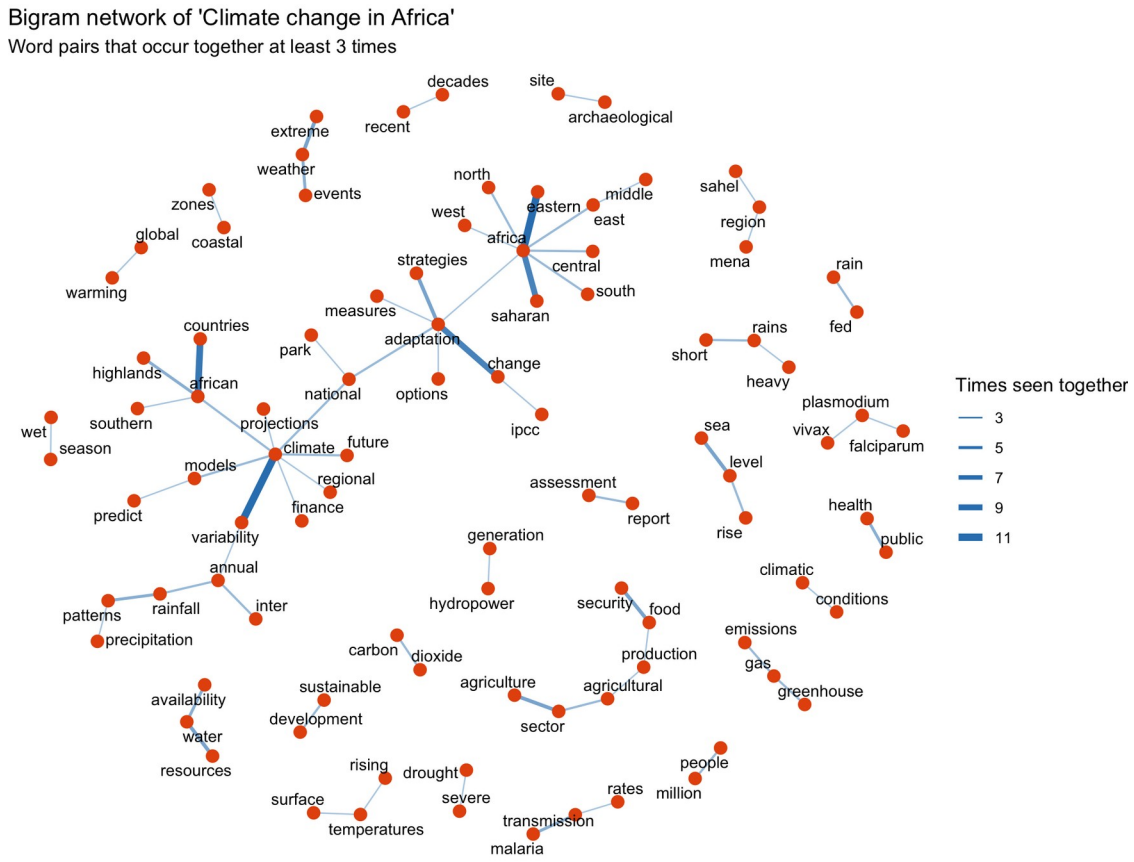
Section	Key sentence
Introduction	Climate change is causing increasingly erratic rainfall patterns, more frequent extreme weather events such as droughts and floods, and rising sea-surface temperatures across the continent.
Greenhouse gas emissions	Africa's per person greenhouse gas emissions are low compared to other continents.
Impacts	It is estimated that by 2030, the people of Africa will be exposed to a rise in sea level due to an increase in temperature.
Socioeconomic impacts	The impacts include higher temperatures, droughts, changing rainfall patterns, and increased climate variability.
Water scarcity	Climate change, and resulting in increased temperatures, storms, droughts, and rising sea levels, will affect the incidence and distribution of infectious disease across the globe.
Impacts by region	The global mean temperature in this region is to increase by 1.5 °C to 2 °C.
Adaptation	In Eastern Africa adaptation options are varied, including improving use of climate information, actions in the agriculture and livestock sector, and in the water sector.
Society and culture	This pattern of parachute science... prevents local

Section	Key sentence
	researchers from doing groundbreaking work, because they do not have the funding...

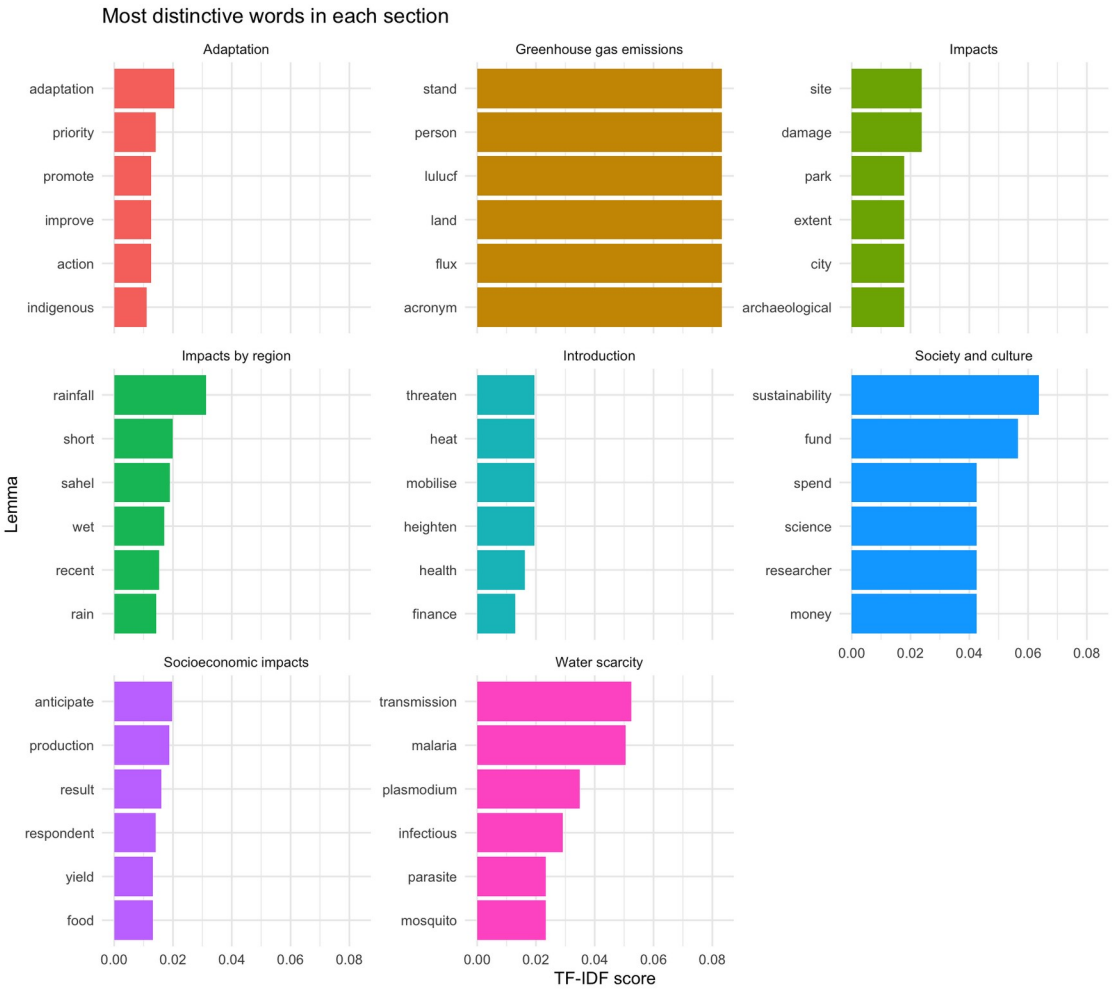
Visualisations



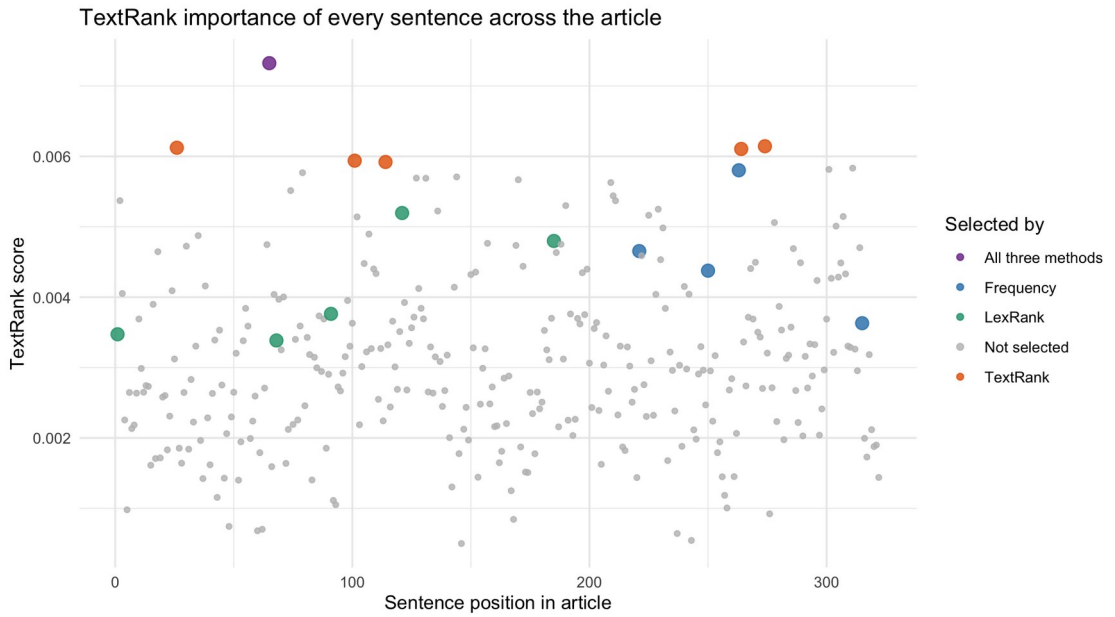
Top 20 content words



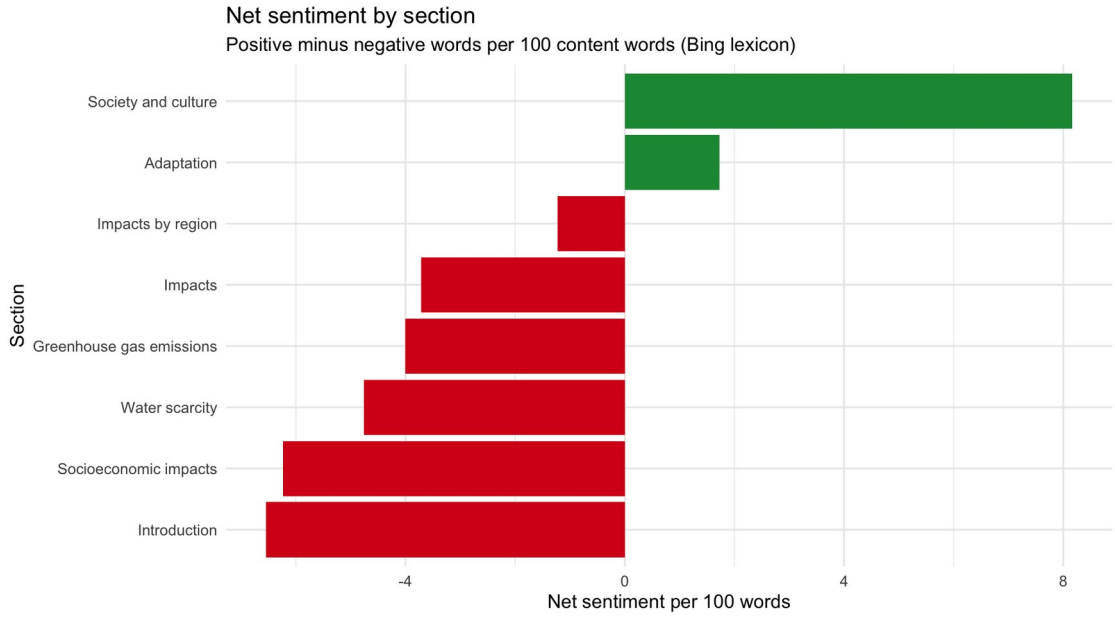
Bigram network



Most distinctive words per section (TF-IDF)



TextRank score of every sentence, coloured by which method selected it



Net sentiment by section

The code for all five charts is in the notebook. As an example, the bigram network:

```
bigrams <- sentences %>%
  unnest_tokens(bigram, sentence, token = "ngrams", n = 2) %>%
  separate(bigram, c("w1", "w2"), sep = " ") %>%
  filter(!w1 %in% stop_words$word, !w2 %in% stop_words$word,
         !str_detect(w1, "\\d"), !str_detect(w2, "\\d")) %>%
  count(w1, w2, sort = TRUE) %>%
  filter(!(w1 == "climate" & w2 == "change"))   # the page title, 101 times, would swamp the
graph

bigrams %>%
```

```
  filter(n >= 3) %>%
  graph_from_data_frame() %>%
  ggraph(layout = "fr") +
  geom_edge_link(aes(edge_alpha = n, edge_width = n), colour = "#3182BD", show.legend = TRUE) +
  geom_node_point(colour = "#E6550D", size = 3) +
  geom_node_text(aes(label = name), repel = TRUE, size = 3.3) +
  scale_edge_width(range = c(0.4, 2), name = "Times seen together") +
  scale_edge_alpha(range = c(0.4, 1), guide = "none") +
  labs(title = "Bigram network of 'Climate change in Africa'",
       subtitle = "Word pairs that occur together at least 3 times") +
  theme_void()
```

Findings

What the article is about. With the title words removed, the most frequent lemmas are *adaptation* (64), *increase* (54), *rainfall* (44), *water* (41) and *temperature* (37). The article is less about the science of warming and more about how a hotter, less predictable climate reaches people through water and rain, and how Africa responds. *Agriculture*, *drought*, *food* and *flood* in the top 20 show that farming and food supply are the main channel of harm.

Main idea clusters. The bigram network splits into a weather cluster (*extreme weather events*, *rainfall patterns*, *sea level rise*), a farming cluster (*agriculture sector*, *food security*, *rain fed*), a health cluster (*malaria transmission*, *plasmodium falciparum*, *public health*), a policy cluster (*national adaptation*, *adaptation strategies*, *climate finance*) and a regional cluster (*eastern*, *saharan*, *north*, *central africa*). Together they work as a table of contents built from the language alone.

What each section adds. TF-IDF shows the sections are well separated. *Water scarcity* is really about disease (*malaria*, *plasmodium*, *mosquito*, *parasite*), which the heading does not suggest. *Impacts by region* is about rainfall seasons in the *Sahel*, and *Society and culture* is about who funds and does the research.

How the summaries differ. The frequency method favours long sentences full of common terms and picked four policy sentences from Adaptation, so it reads like a list of programmes. TextRank favours short, general sentences that share words with many others, and centres on agriculture and rainfall. LexRank picked the opening sentence, which says Africa is among the most vulnerable regions while producing under 4% of global emissions, because it combines rare and central ideas. The position chart shows high scoring sentences spread across the whole article, so a summary that just takes the first few sentences would miss a lot. The section by section digest is the most complete summary for a reader.

Tone. Sentiment is negative where the article describes the problem (Introduction -6.5 and Socioeconomic impacts -6.2 net words per 100) and positive where it describes responses (Adaptation +1.7, Society and culture +8.2). The article follows a problem then response structure. The most common negative words are *risk*, *drought* and *vulnerable*.

Limitations. The summaries are extractive, so they can only reuse sentences already on the page. The Bing lexicon is general purpose and treats words like *well* and *like* as positive even when they are not used that way.

Question 4: ETL with R and PostgreSQL

The notebook is `q4-etl/q4_etl_books.Rmd`. The SQL lives in `q4-etl/sql/`.

4.1 Use case and scraping

Use case. A small online bookshop wants to monitor a competitor's catalogue: how books are priced, whether better reviewed books are priced or stocked differently, and which categories hold the most stock value. The competitor site is books.toscrape.com, a public catalogue made for scraping practice. It has no `robots.txt` (the request returns 404), so no path is disallowed. The scraper still waits half a second between requests and caches every page in `data/raw/q4_pages/`.

The scraper reads the first 5 listing pages, follows the link to every book, and keeps every field as raw text exactly as it appears on the site. This gives **100 records** with a textual attribute (title, category, description) and numerical attributes that have to be inferred from text (price, star rating, stock count).

```
base_url <- "https://books.toscrape.com/catalogue/"
cache_dir <- "../data/raw/q4_pages"
dir.create(cache_dir, showWarnings = FALSE, recursive = TRUE)

get_page <- function(url) {
  file <- file.path(cache_dir, paste0(str_replace_all(str_remove(url, "^https?://"), "[^A-Za-z0-
9]+", "_"), ".html"))
  if (!file.exists(file) || params$refresh) {
    Sys.sleep(0.5)  # polite delay between requests
    resp <- GET(url, user_agent("itbda-student-etl/1.0 (educational project)"))
    stop_for_status(resp)
    writeLines(content(resp, as = "text", encoding = "UTF-8"), file, useBytes = TRUE)
  }
  read_html(file, encoding = "UTF-8")
}

# Listing page: title as shown in the grid (often cut short) and the link to the detail page
scrape_listing <- function(page_no) {
  page  <- get_page(paste0(base_url, "page-", page_no, ".html"))
  cards <- html_elements(page, "article.product_pod")
  tibble(
    listing_page = page_no,
    title_listing = html_text(html_element(cards, "h3 a")),
    url = paste0(base_url, html_attr(html_element(cards, "h3 a"), "href"))
  )
}

# Detail page: everything kept as raw text, no cleaning here
scrape_book <- function(url) {
  page  <- get_page(url)
  table <- html_table(html_element(page, "table.table-striped"))
  field <- function(name) table$X2[table$X1 == name]
  tibble(
    url              = url,
    title_raw        = html_text(html_element(page, ".product_main h1")),
    price_raw        = html_text(html_element(page, ".product_main .price_color")),
    rating_raw       = html_attr(html_element(page, ".product_main .star-rating"), "class"),
    availability_raw = html_text(html_element(page, ".product_main .availability")),
    category_raw     = html_text(html_elements(page, ".breadcrumb li"))[3],
    description_raw  = html_text(html_element(page, "#product_description + p")),
    upc_raw          = field("UPC"),
    product_type_raw = field("Product Type"),
    price_excl_tax_raw = field("Price (excl. tax)"),
    tax_raw          = field("Tax"),
    reviews_raw      = field("Number of reviews")
  )
}
```

```
base_url <- "https://books.toscrape.com/catalogue/"
cache_dir <- "../data/raw/q4_pages"
dir.create(cache_dir, showWarnings = FALSE, recursive = TRUE)

get_page <- function(url) {
  file <- file.path(cache_dir, paste0(str_replace_all(str_remove(url, "^https?://"), "[^A-Za-z0-
9]+", "_"), ".html"))
  if (!file.exists(file) || params$refresh) {
    Sys.sleep(0.5)  # polite delay between requests
    resp <- GET(url, user_agent("itbda-student-etl/1.0 (educational project)"))
    stop_for_status(resp)
    writeLines(content(resp, as = "text", encoding = "UTF-8"), file, useBytes = TRUE)
  }
  read_html(file, encoding = "UTF-8")
}

# Listing page: title as shown in the grid (often cut short) and the link to the detail page
scrape_listing <- function(page_no) {
  page  <- get_page(paste0(base_url, "page-", page_no, ".html"))
  cards <- html_elements(page, "article.product_pod")
  tibble(
    listing_page = page_no,
    title_listing = html_text(html_element(cards, "h3 a")),
    url = paste0(base_url, html_attr(html_element(cards, "h3 a"), "href"))
  )
}

# Detail page: everything kept as raw text, no cleaning here
scrape_book <- function(url) {
  page  <- get_page(url)
  table <- html_table(html_element(page, "table.table-striped"))
  field <- function(name) table$X2[table$X1 == name]
  tibble(
    url              = url,
    title_raw        = html_text(html_element(page, ".product_main h1")),
    price_raw        = html_text(html_element(page, ".product_main .price_color")),
    rating_raw       = html_attr(html_element(page, ".product_main .star-rating"), "class"),
    availability_raw = html_text(html_element(page, ".product_main .availability")),
    category_raw     = html_text(html_elements(page, ".breadcrumb li"))[3],
    description_raw  = html_text(html_element(page, "#product_description + p")),
    upc_raw          = field("UPC"),
    product_type_raw = field("Product Type"),
    price_excl_tax_raw = field("Price (excl. tax)"),
    tax_raw          = field("Tax"),
    reviews_raw      = field("Number of reviews")
  )
}
```

The raw data contains the spurious, inconsistent and incomplete values typical of real web data:

Issue in the raw data	Rows affected
Listing title cut short with "..."	66
Price stored as text with a currency sign	100
Rating stored as a CSS class word ("star-rating Three")	100
Stock count buried in padded text ("In stock (22 available)")	100
Placeholder category ("Default" or "Add a comment"), really missing	14
Description ends with the "...more" teaser	92
Description text repeated inside itself	91
Product type, tax and review count identical for every row (spurious columns)	100

4.2 Cleaning, denoising and imputation

Regex parsing. Numbers and labels are pulled out of the text with regular expressions: price with `[0-9]+\.[0-9]{2}`, rating with `(?<=star-rating )[A-Za-z]+` mapped from words to 1 to 5, and stock with `(?<=\()[0-9]+(?= available)`. The UPC is validated against `^[0-9a-f]{16}$`.

```
rating_words <- c(One = 1, Two = 2, Three = 3, Four = 4, Five = 5)

books <- books_raw %>%
  transmute(
    upc          = str_squish(upc_raw),
    title        = str_squish(title_raw),
    title_was_truncated = str_detect(title_listing, "\\.\\.\\.$"),
    category     = str_squish(category_raw),
    price_gbp    = as.numeric(str_extract(price_raw, "[0-9]+\\.[0-9]{2}")),
    rating       = unname(rating_words[str_extract(rating_raw, "(?<=star-rating )[A-Za-z]+")]),
    stock_count  = as.integer(str_extract(availability_raw, "(?<=\\()[0-9]+(?= available)")),
    description  = str_squish(description_raw),
    url,
    scraped_at
  )

stopifnot(all(str_detect(books$upc, "^[0-9a-f]{16}$")))
summary(select(books, price_gbp, rating, stock_count))
```

Spurious data and duplicates. Product type ("Books"), tax ("£0.00") and number of reviews ("0") never change, so they carry no information and are dropped. Rows are de-duplicated on the UPC (no duplicates were found in this scrape).

Denoising. The repeated preview inside each description is removed with a match on the opening 60 characters, and the "...more" teaser is stripped. The average description shrinks from 225 to 167 words, and no repeats or teasers remain.

```
dedupe_description <- function(x) {
  start <- str_sub(x, 1, 60)
  second <- str_locate_all(x, fixed(start))     # every place the opening appears
  map2_chr(x, second, function(txt, pos) {
    if (is.na(txt)) return(NA_character_)
    if (nrow(pos) >= 2) str_sub(txt, pos[2, 1]) else txt
  })
}

books <- books %>%
  mutate(description = description %>%
           dedupe_description() %>%
           str_remove("\\s*\\.\\.\\.more$") %>%
           str_squish())

tibble(
  avg_words_raw   = mean(str_count(books_raw$description_raw, "\\S+"), na.rm = TRUE),
  avg_words_clean = mean(str_count(books$description, "\\S+"), na.rm = TRUE),
  still_repeating = sum(str_detect(books$description, "^(.{60}).*\\1"), na.rm = TRUE),
  still_has_more  = sum(str_detect(books$description, "\\.\\.\\.more$"), na.rm = TRUE)
)
```

Missing values and imputation. "Default" and "Add a comment" are what the site shows when no genre was set, so they are converted to missing (14 books). They are imputed with a **k nearest neighbour** rule on the book descriptions: each description becomes a TF-IDF vector, and a missing category takes the most common category among the 3 most similar labelled books (cosine similarity). A leave one out test on labelled books gives 26% accuracy against a 16% majority class baseline. That is better than guessing but not reliable enough for every book, so a similarity threshold of 0.05 sends weak matches to "Uncategorised". In the end 4 books received a genre and 10 were labelled "Uncategorised". Every imputed row is flagged in `category_imputed`. Stock counts

that fail to parse would be imputed with the **median of their category**, and missing descriptions with a **constant**. Neither happened in this scrape, but the rules stay in the pipeline for future runs.

```
library(tidytext)

desc_vectors <- books %>%
  mutate(doc = row_number()) %>%
  unnest_tokens(word, description) %>%
  anti_join(stop_words, by = "word") %>%
  filter(!str_detect(word, "\\d")) %>%
  count(doc, word) %>%
  bind_tf_idf(word, doc, n) %>%
  select(doc, word, tf_idf) %>%
  pivot_wider(names_from = word, values_from = tf_idf, values_fill = 0)

m <- as.matrix(desc_vectors[, -1]); rownames(m) <- desc_vectors$doc
m <- m / sqrt(rowSums(m^2))
cos_sim <- m %*% t(m)

knn_category <- function(i, labelled, k = 3, min_sim = 0.05) {
  sims <- cos_sim[as.character(i), as.character(labelled)]
  top  <- head(sort(sims, decreasing = TRUE), k)
  if (max(top) < min_sim) return(c(category = "Uncategorised", similarity = max(top)))
  votes <- table(books$category[as.integer(names(top))])
  c(category = names(votes)[which.max(votes)], similarity = max(top))
}

labelled <- which(!is.na(books$category))
missing  <- which(is.na(books$category))

# How reliable is it? Leave one out test on books whose category is known
loo <- sapply(labelled, function(i) knn_category(i, setdiff(labelled, i))["category"])
loo_accuracy <- mean(loo == books$category[labelled])
baseline <- max(table(books$category[labelled])) / length(labelled)
tibble(knn_accuracy = round(loo_accuracy, 2), majority_class_baseline = round(baseline, 2))

imputed <- t(sapply(missing, knn_category, labelled = labelled))
books$category_imputed <- FALSE
books$category[missing] <- imputed[, "category"]
books$category_imputed[missing] <- TRUE

books %>% filter(category_imputed) %>%
  transmute(title = str_trunc(title, 50), imputed_category = category,
            best_similarity = round(as.numeric(imputed[, "similarity"]), 3))
```

An IQR check found no price or stock outliers (prices £10.16 to £58.11, stock 16 to 22 units). Finally, a `price_band` column was derived and the clean data was exported to `data/processed/q4_books_clean.csv` (100 rows, 15 columns, no missing values).

```
books_clean <- books %>%
  select(upc, title, title_was_truncated, category, category_imputed, price_gbp, price_band,
         rating, stock_count, stock_imputed, description, description_imputed,
         description_words, url, scraped_at)

write_csv(books_clean, "../data/processed/q4_books_clean.csv", na = "NA")
glimpse(books_clean)
```

4.3 Loading into PostgreSQL and analysis

The table is created with a primary key and CHECK constraints (rating 1 to 5, price above 0, stock not negative), which act as a final quality gate. The CSV is imported with `\copy`:

```
-- Question 4.3: create the table and import the cleaned CSV
-- Run from the q4-etl folder:  psql -d itbda_etl -f sql/01_create_and_load.sql
```

```
DROP TABLE IF EXISTS books;

CREATE TABLE books (
    upc                 CHAR(16)      PRIMARY KEY,
    title               TEXT          NOT NULL,
    title_was_truncated BOOLEAN       NOT NULL,
    category            TEXT          NOT NULL,
    category_imputed    BOOLEAN       NOT NULL,
    price_gbp           NUMERIC(6, 2) NOT NULL CHECK (price_gbp > 0),
    price_band          TEXT          NOT NULL,
    rating              SMALLINT      NOT NULL CHECK (rating BETWEEN 1 AND 5),
    stock_count         INTEGER       NOT NULL CHECK (stock_count >= 0),
    stock_imputed       BOOLEAN       NOT NULL,
    description         TEXT          NOT NULL,
    description_imputed BOOLEAN       NOT NULL,
    description_words   INTEGER       NOT NULL,
    url                 TEXT          NOT NULL,
    scraped_at          TIMESTAMP     NOT NULL
);

\copy books FROM '../data/processed/q4_books_clean.csv' WITH (FORMAT csv, HEADER true, NULL 'NA')

-- Quick checks that the import is complete
SELECT COUNT(*)                         AS rows_loaded,
       COUNT(DISTINCT category)         AS categories,
       SUM(category_imputed::int)       AS imputed_categories,
       SUM(description_imputed::int)    AS imputed_descriptions,
       MIN(price_gbp)                   AS min_price,
       MAX(price_gbp)                   AS max_price
FROM books;
```

```
DROP TABLE
CREATE TABLE
COPY 100
 rows_loaded | categories | imputed_categories | imputed_descriptions | min_price | max_price
-------------+------------+--------------------+----------------------+-----------+-----------
         100 |         28 |                 14 |                    0 |     10.16 |     58.11
```

Investigation 1: does a better rating mean a higher price or more stock?

```
-- Investigation 1: do better rated books cost more or carry more stock?
-- Summarises the catalogue by star rating and adds the overall
-- correlation of rating with price and with stock.
WITH overall AS (
    SELECT ROUND(CORR(rating, price_gbp)::numeric, 3)   AS corr_rating_price,
           ROUND(CORR(rating, stock_count)::numeric, 3) AS corr_rating_stock,
           COUNT(*)                                     AS total_books
    FROM books
)
SELECT b.rating,
       COUNT(*)                                          AS books,
       ROUND(100.0 * COUNT(*) / o.total_books, 1)        AS pct_of_catalogue,
       ROUND(AVG(b.price_gbp), 2)                        AS avg_price_gbp,
       ROUND(AVG(b.stock_count), 1)                      AS avg_stock,
       SUM(b.price_gbp * b.stock_count)                  AS stock_value_gbp,
       o.corr_rating_price,
       o.corr_rating_stock
FROM books b
CROSS JOIN overall o
GROUP BY b.rating, o.total_books, o.corr_rating_price, o.corr_rating_stock
ORDER BY b.rating;
```

```
 rating | books | pct_of_catalogue | avg_price_gbp | avg_stock | stock_value_gbp |
corr_rating_price | corr_rating_stock
--------+-------+------------------+---------------+-----------+-----------------
+-------------------+-------------------
      1 |    22 |             22.0 |         35.52 |      17.2 |        13478.61 |
-0.122 |             0.064
      2 |    19 |             19.0 |         35.91 |      16.8 |        11455.99 |
-0.122 |             0.064
```

```
      3 |    22 |             22.0 |         36.84 |      17.3 |        14171.88 |
-0.122 |             0.064
      4 |    18 |             18.0 |         33.98 |      16.9 |        10400.60 |
-0.122 |             0.064
      5 |    19 |             19.0 |         30.01 |      17.5 |        10100.47 |
-0.122 |             0.064
(5 rows)
```

Interpretation. Ratings are spread almost evenly (18% to 22% at each star level). Price hardly changes with rating: one star books average £35.52 and five star books £30.01, and the correlation is only -0.12, so the best reviewed books are, if anything, slightly cheaper. Stock is flat at about 17 units at every level (correlation 0.06). The competitor does not price or stock by review quality, which means highly rated books we sell at a premium can be found cheaper there. With only 100 books this is weak evidence and should be rechecked on the full 1,000 title catalogue.

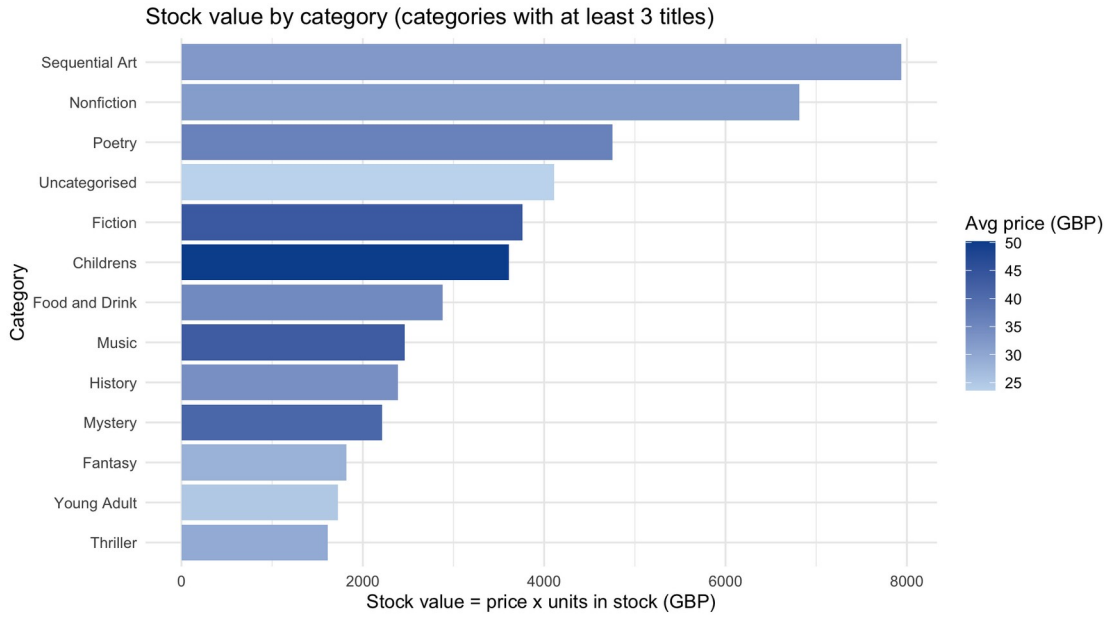
Investigation 2: which categories hold the most stock value?

```sql
-- Investigation 2: which categories hold the most stock value, and what is
-- the top priced title in each? Uses window functions to rank categories
-- and to pick the most expensive book per category.
WITH ranked_books AS (
    SELECT category,
           title,
           price_gbp,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY price_gbp DESC) AS price_rank
    FROM books
),
category_summary AS (
    SELECT category,
           COUNT(*)                              AS books,
           ROUND(AVG(price_gbp), 2)              AS avg_price_gbp,
           ROUND(AVG(rating), 2)                 AS avg_rating,
           SUM(stock_count)                      AS units_in_stock,
           SUM(price_gbp * stock_count)          AS stock_value_gbp,
           SUM(category_imputed::int)            AS imputed_rows
    FROM books
    GROUP BY category
    HAVING COUNT(*) >= 3
)
SELECT RANK() OVER (ORDER BY c.stock_value_gbp DESC)                    AS value_rank,
       c.category,
       c.books,
       c.avg_price_gbp,
       c.avg_rating,
       c.units_in_stock,
       c.stock_value_gbp,
       ROUND(100.0 * c.stock_value_gbp / SUM(c.stock_value_gbp) OVER (), 1) AS pct_of_value,
       c.imputed_rows,
       r.title                                                          AS top_priced_title,
       r.price_gbp                                                      AS top_price_gbp
FROM category_summary c
JOIN ranked_books r
  ON r.category = c.category AND r.price_rank = 1
ORDER BY value_rank;
```

Rank	Category	Books	Avg price	Avg rating	Units	Stock value (£)	% of value
1	Sequential Art	15	32.42	2.80	243	7,937.19	17.2
2	Nonfiction	13	31.82	3.15	214	6,811.10	14.8
3	Poetry	7	36.24	2.57	129	4,750.35	10.3
4	Uncategorised	10	23.60	2.80	176	4,109.36	8.9
5	Fiction	5	43.87	3.80	85	3,757.77	8.2

Rank	Category	Books	Avg price	Avg rating	Units	Stock value (£)	% of value
6	Childrens	4	50.20	2.00	72	3,609.73	7.8
7	Food and Drink	5	34.89	2.20	83	2,882.44	6.3

(Showing the top 7 of 13 categories with at least three titles. The query also returns the most expensive title per category, for example "Slow States of Collapse: Poems" at £57.31 in Poetry. The full output is in `q4-etl/sql/output/`.)



Stock value by category

Interpretation. Sequential Art (graphic novels) and Nonfiction hold about 32% of the stock value. Their lead comes from the number of titles, not from price, since their averages (£32.42 and £31.82) are below Fiction (£43.87) and Childrens (£50.20). Childrens books are the most expensive group but the worst rated (2.0 stars), which looks like a pricing weakness. Fiction combines a high price with the best rating (3.8), so it is the competitor's strongest category. The "Uncategorised" group, which exists only because of missing source data, still ranks fourth with almost 9% of stock value. Poor category data would hide that share of the catalogue from category reports, which is why the imputation flags matter.

4.4 GitHub repository

The private repository **https://github.com/HappyDevs1/big-data-text-mining-etl** contains:

- **Documentation:** the root `README.md` (structure and how to run everything), a README per question folder, and this report (`docs/ITBDA4-14_Project2_Report.docx`, built from `docs/report_template.md` by `docs/build_report.py`).
- **Datasets:** `data/raw/` (the two zips from the brief, the Wikipedia snapshot, the cached book pages and the raw scraped CSV), `data/extracted/` (unzipped documents) and `data/processed/` (cleaned outputs of Questions 2, 3 and 4).

- **Notebooks:** one R Markdown notebook per R question with its rendered HTML, showing the flow from raw data to results. For Question 1 the Java project, run scripts and `evidence/` folder (logs, results and screenshots) play the same role.
- **History:** the work was committed in small steps, so the commit log shows how the project was built.

References

Blei, D.M., Ng, A.Y. and Jordan, M.I. (2003) 'Latent Dirichlet allocation', *Journal of Machine Learning Research*, 3, pp. 993-1022.

Books to Scrape (2026) *All products*. Available at: https://books.toscrape.com (Accessed: 12 November 2026).

Erkan, G. and Radev, D.R. (2004) 'LexRank: Graph-based lexical centrality as salience in text summarization', *Journal of Artificial Intelligence Research*, 22, pp. 457-479.

Luhn, H.P. (1958) 'The automatic creation of literature abstracts', *IBM Journal of Research and Development*, 2(2), pp. 159-165.

Matanga, N.Y. (2026) *ITBDA4-14 Project 2 brief*. Eduvos.

Mihalcea, R. and Tarau, P. (2004) 'TextRank: Bringing order into texts', *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pp. 404-411.

Silge, J. and Robinson, D. (2017) *Text mining with R: A tidy approach*. Sebastopol, CA: O'Reilly Media.

The Apache Software Foundation (2026) *Apache Hadoop MapReduce tutorial*. Available at: https://hadoop.apache.org/docs/stable/ (Accessed: 12 November 2026).

Wikipedia (2026) *Climate change in Africa*. Available at: https://en.wikipedia.org/wiki/Climate_change_in_Africa (Accessed: 12 November 2026).

AI Evidence Section

AI Disclosure Appendix

AI tool used	Purpose / how it was used	How I verified or adapted the output

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance with institutional policy.

Student Signature(s)	Date
Happy Mahlangu	**13 Nov 2026**